



Praktikum Pemrograman Mobile
Pertemuan 1 :
**Inisialisasi Project Android
&
Implementasi Function Sederhana**

Disusun oleh :
Azis Amirulbahar, S.P.d., M.T.I.

**Program Studi Informatika
Fakultas Teknik
Universitas Jenderal Soedirman
2026**



DAFTAR ISI

A. Pengantar	3
B. Tujuan Praktikum	7
C. Alat dan Bahan	8
D. Dasar Teori	8
E. Sekilas tentang <i>Android Studio</i>	8
F. Mengetahui Android SDK	10
G. Membuat Project Baru pada Android Studio.....	11
H. Mengetahui Struktur Project Android	15
I. Mengganti Icon Aplikasi	17
J. Membuat Function.....	22
K. Menjalankan aplikasi	34
L. Bantuan Artificial Intellegence (AI) Gemini untuk Membantu Memecahkan Masalah Penulisan Kode.....	35

A. Pengantar

Selamat datang di Praktikum Pemrograman Mobile. Berbeda dengan tahun ajaran 2025/2026, yang mengimplementasikan antarmuka (UI) dibangun secara terpisah menggunakan berkas **XML** dan dimanipulasi dengan pendekatan imperatif (mencari ID komponen dan mengubahnya satu per satu dari dalam kode) menggunakan bahasa pemrograman Kotlin. Mulai tahun ini materi akan mengadopsi paradigma **Declarative UI** dengan **Jetpack Compose** yang merupakan *toolkit* UI modern yang direkomendasikan oleh Google, kita hanya perlu mendeskripsikan *bagaimana* bentuk tampilan seharusnya berdasarkan status (*state*) data saat ini. Seluruh antarmuka dan logika aplikasi kini ditulis dalam satu bahasa yang sama: **Kotlin**. Tanpa adanya pemisahan antara XML dan kelas logika, proses perancangan aplikasi menjadi jauh lebih intuitif, cepat, dan relevan dengan tren standar industri perangkat lunak.

Kotlin adalah bahasa pemrograman modern, diketik secara statis (*statically typed*), dan lintas platform yang dikembangkan oleh JetBrains. Sejak tahun 2019, Google secara resmi menetapkan Kotlin sebagai bahasa pemrograman utama (*first-class language*) yang paling direkomendasikan untuk pengembangan aplikasi Android, menggeser posisi Java. Kotlin menawarkan sintaks yang jauh lebih ringkas, ekspresif, dan aman. Salah satu keunggulan utamanya adalah fitur *null-safety* bawaan yang secara proaktif mencegah *NullPointerException*—sebuah celah *bug* yang sering menjadi momok dalam pengembangan aplikasi berbasis Java.

Secara teknis, Kotlin berjalan di atas *Java Virtual Machine* (JVM). Hal ini memberikan Kotlin **interoperabilitas 100% dengan Java**, yang berarti kode Kotlin dan Java dapat dipanggil secara dua arah dan digunakan bersamaan di dalam satu proyek yang sama tanpa adanya kendala kompilasi.



Gambar 1. Logo Kotlin

(Sumber: <https://kotlinlang.org/docs/kotlin-brand-assets.html>)

Sebagai bahasa pemrograman yang dirancang untuk mengatasi kelemahan pada bahasa-bahasa pendahulunya, Kotlin memiliki keunggulan sebagai berikut:

- **Ringkas (*Concise*):** Kotlin secara drastis mengurangi penulisan kode berulang (*boilerplate*). Fitur-fitur otomatis seperti pembentukan *getter*, *setter*, dan *data classes* membuat berkas kode menjadi jauh lebih pendek, efisien, dan mudah dipelihara.
- **Aman (*Safe*):** Kotlin dirancang dengan fitur *Null-Safety* terintegrasi. Sistem kompilator Kotlin secara proaktif membedakan variabel yang boleh bernilai kosong (*nullable*) dan yang tidak (*non-nullable*), sehingga mencegah terjadinya *NullPointerException*—sebuah celah *bug* yang sering menyebabkan aplikasi Java berhenti mendadak (*crash*).
- **Ekspresif (*Expressive*):** Kotlin mendukung paradigma pemrograman berorientasi objek maupun fungsional. Bahasa ini dilengkapi fitur-fitur tingkat lanjut seperti *lambdas*, *higher-order functions*, dan *coroutines* (untuk mengeksekusi proses asinkron yang kompleks dengan cara yang sederhana).
- **Modern dan Adaptif:** Selain untuk Android, ekosistem Kotlin kini telah berkembang pesat (*Kotlin Multiplatform*) sehingga dapat digunakan untuk pengembangan aplikasi sisi peladen (*server-side*), aplikasi web, hingga aplikasi desktop.

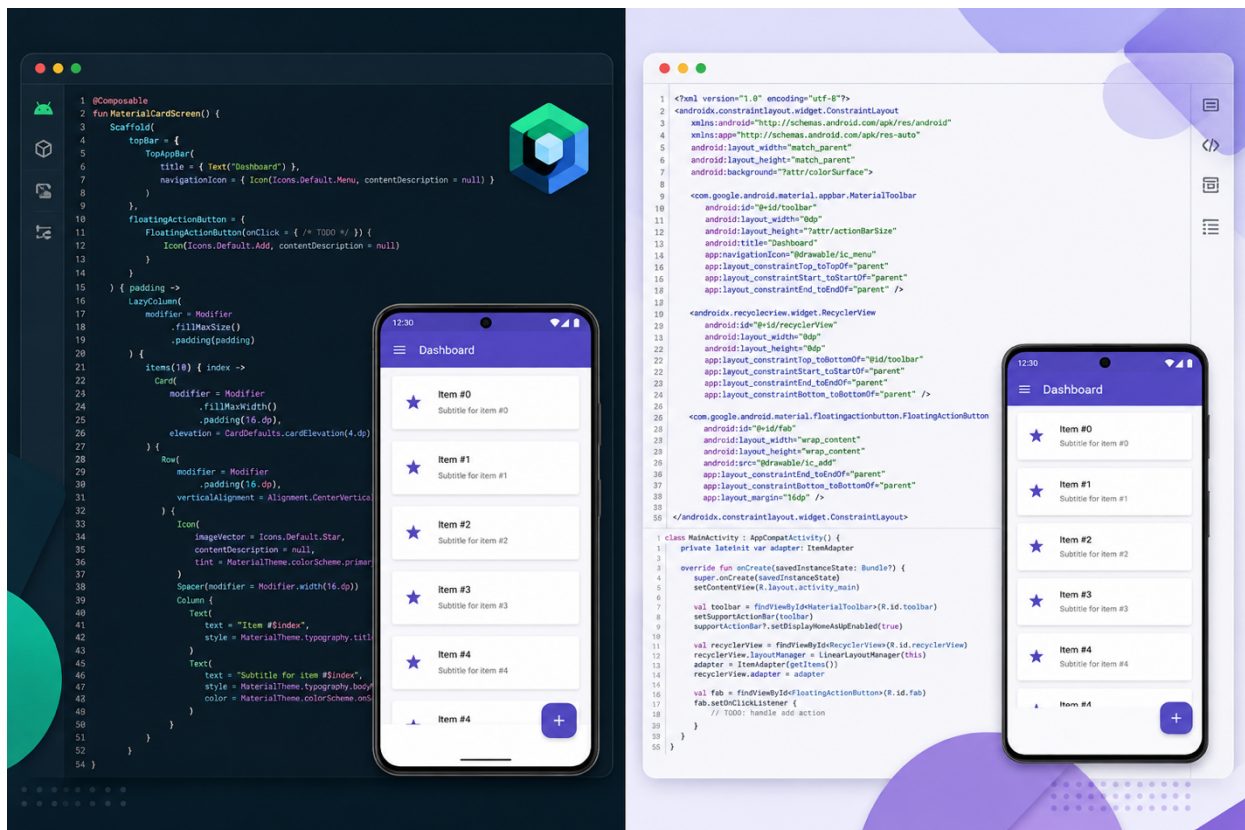
Jetpack Compose adalah *toolkit* (perangkat alat) pengembangan modern resmi dari Android yang dirancang untuk membangun antarmuka pengguna (UI) aplikasi secara *native*. Menggantikan sistem hierarki tampilan tradisional yang berbasis XML, Jetpack Compose sepenuhnya memanfaatkan bahasa pemrograman Kotlin dan mengadopsi paradigma **Declarative UI** (Antarmuka Deklaratif).



Gambar 2. Logo Jetpack Compose

(Sumber: <https://android-developers.googleblog.com/2022/05/whats-new-in-jetpack-compose.html>)

Secara konseptual, alih-alih merangkai komponen UI secara visual lalu menghubungkannya dengan logika bisnis melalui referensi ID, *Jetpack Compose* menggunakan fungsi-fungsi Kotlin (yang ditandai dengan anotasi `@Composable`) untuk secara langsung mendeskripsikan bentuk antarmuka berdasarkan status (*state*) data saat ini.



Gambar 3. Perbandingan Baris Kode Antara Implementasi UI dengan Jetpack Compose (Kiri) dan XML & Kotlin (Kanan)

Gambar di atas menunjukkan perbandingan antara dua pendekatan dalam pengembangan antarmuka pengguna (User Interface/UI) pada aplikasi Android. Bagian kiri menggambarkan pendekatan **Declarative UI menggunakan Jetpack Compose**, di mana antarmuka dibangun secara deklaratif dengan mendeskripsikan tampilan berdasarkan state atau kondisi aplikasi. Pengembang cukup menentukan *apa* yang ingin ditampilkan, sedangkan Jetpack Compose akan menangani proses pembaruan antarmuka ketika terjadi perubahan data atau state.

Sementara itu, bagian kanan menggambarkan pendekatan **Imperative UI menggunakan XML dan Kotlin**, di mana struktur antarmuka didefinisikan terlebih dahulu menggunakan file XML, kemudian komponen antarmuka dikontrol dan dimanipulasi melalui kode Kotlin. Pada pendekatan ini, pengembang secara eksplisit menentukan *bagaimana* komponen antarmuka dibuat, diakses, dan diperbarui.

Beberapa karakteristik utama dari Jetpack Compose meliputi:

- **Deklaratif dan Reaktif:** Pengembang hanya mendeskripsikan *apa* yang harus ditampilkan layar. Ketika data atau status berubah, kerangka kerja (*framework*) akan secara otomatis memperbarui dan menggambar ulang (*recomposition*) hanya pada bagian antarmuka yang terdampak.
- **Kode Lebih Ringkas:** Menghapus kebutuhan penulisan *boilerplate code* (seperti `findViewById` atau *Adapter* pada `RecyclerView`), sehingga kode jauh lebih ringkas, mudah dibaca, dan lebih cepat untuk ditulis.
- **Satu Bahasa Terpadu:** Seluruh tata letak, desain, animasi, dan logika bisnis ditulis menggunakan satu bahasa pemrograman yang sama, yaitu Kotlin, tanpa perlu lagi berpindah ke berkas XML.
- **Kesesuaian Penuh (*Interoperability*):** Jetpack Compose dirancang untuk dapat bekerja berdampingan dengan pendekatan XML lama. Artinya, komponen Compose dapat disisipkan ke dalam proyek aplikasi Android lama yang masih menggunakan XML, begitu pula sebaliknya.

B. Tujuan Praktikum

1. Mahasiswa memahami perbedaan mendasar antara pendekatan pengembangan Android tradisional (Java/XML) dengan pendekatan modern (Kotlin/Jetpack Compose).
2. Mahasiswa mampu memahami dan mengimplementasikan sintaks dasar Kotlin (variabel, tipe data, fungsi, dan *null-safety*).
3. Mahasiswa memahami konsep *Declarative UI* dan struktur dasar fungsi `@Composable`.
4. Mahasiswa mampu membuat tata letak (*layout*) dasar menggunakan `Column`, `Row`, dan `Box`.

C. Alat dan Bahan

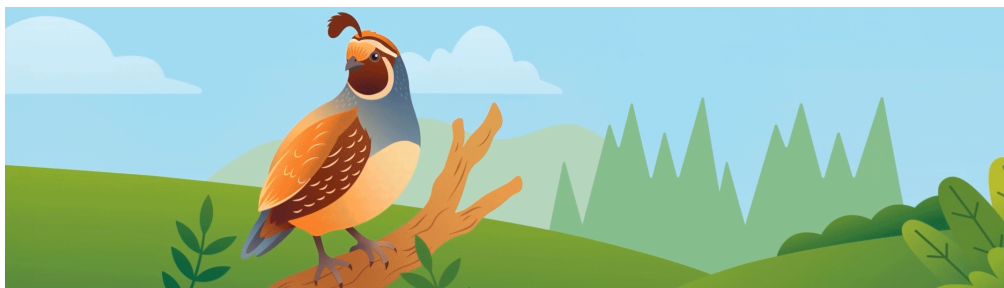
1. Perangkat keras (PC/Laptop) dengan spesifikasi minimum sesuai standar Android Studio.
2. Android Studio (versi terbaru yang sudah mendukung Jetpack Compose secara bawaan).
3. Emulator Android (AVD) atau perangkat fisik Android yang terhubung melalui *debugging* (USB/Wi-Fi).

D. Dasar Teori

1. **Pengenalan Kotlin:**
 - Konsep dasar (Ringkas, Ekspresif, dan Aman).
2. **Pergeseran Paradigma UI:**
 - Imperative UI (XML) vs Declarative UI (Jetpack Compose).
3. **Anatomi Jetpack Compose:**
 - Anotasi `@Composable` dan `@Preview`.
 - Penggunaan Modifier untuk mengatur tampilan komponen (ukuran, warna, *padding*).

E. Sekilas tentang *Android Studio*

Android Studio merupakan IDE (*Integrated Development Environment*) yang secara resmi diperkenalkan oleh Google untuk mengembangkan perangkat lunak berbasis sistem operasi android. Penamaan Android Studio menggunakan pola alfabet dengan inisial nama – nama hewan. Pada saat modul ini dibuat pada bulan Agustus 2026, versi android telah memasuki versi Q dengan inisial *Quail* (burung puyuh).



Gambar 4. Simbol Versi Quail pada Android Studio

Instalasi Android Studio dapat diperoleh pada tautan berikut: <https://developer.android.com/studio>, sedangkan fungsi dan tata letak menu Android Studio secara lengkap dapat dipelajari melalui tautan berikut: <https://developer.android.com/studio/intro/user-interface> .

F. Mengetahui Android SDK

Android SDK (Software Development Kit) adalah seperangkat alat pengembangan resmi yang disediakan oleh Google untuk membangun aplikasi Android. SDK berisi pustaka (*library*), API (*Application Programming Interface*), *compiler*, *debugger*, serta emulator yang memudahkan developer menulis, menguji, dan menjalankan aplikasi android.

Tanpa SDK, aplikasi Android tidak dapat dibuat maupun dijalankan, karena SDK menyediakan "jembatan" antara kode program dengan sistem operasi android.

Fungsi android SDK di antaranya :

1. **Menyediakan API:** *Developer* dapat memanggil fitur android seperti kamera, GPS, *bluetooth*, sensor, dan lain-lain.
2. **Membangun aplikasi:** Dengan *build tools*, kode *Kotlin/Java/Flutter* akan dikompilasi menjadi APK/AAB.
3. **Testing aplikasi:** melakukan uji aplikasi melalui *emulator* atau perangkat fisik.
4. **Debugging:** Dengan *adb* dan *profiler*, developer dapat melacak bug atau masalah performa.
5. **Kompatibilitas:** Mendukung banyak versi android, sehingga aplikasi dapat dibuat agar kompatibel pada berbagai perangkat.

Berikut ini adalah komponen utama dalam SDK android :

Komponen	Deskripsi
SDK Tools	Berisi <i>tool</i> dasar untuk mengembangkan aplikasi, seperti <i>adb (Android Debug Bridge)</i> , <i>fastboot</i> , dan <i>profiler</i> .
SDK Platform Tools	Mendukung fitur spesifik platform Android, misalnya debugging aplikasi pada perangkat fisik.
SDK Build Tools	Berisi <i>compiler</i> , <i>linker</i> , dan utilitas lain untuk membangun (build) aplikasi android.
Android Platform	Paket berisi <i>library</i> inti (API) untuk versi Android tertentu. Contoh: Android 13 API Level 33. Daftar versi android API dapat dilihat pada: https://developer.android.com/tools/releases/platforms

Komponen	Deskripsi
<i>System Images</i>	Digunakan untuk membuat dan menjalankan emulator Android (AVD – <i>Android Virtual Device</i>).
<i>Google Play Services</i>	Library tambahan untuk integrasi layanan <i>Google</i> (<i>Maps, Firebase, Location, dll.</i>).
<i>Documentation & Samples</i>	Dokumentasi resmi dan contoh kode untuk mempelajari teknik pengembangan aplikasi.

Tabel 1. Komponen utama Android SDK

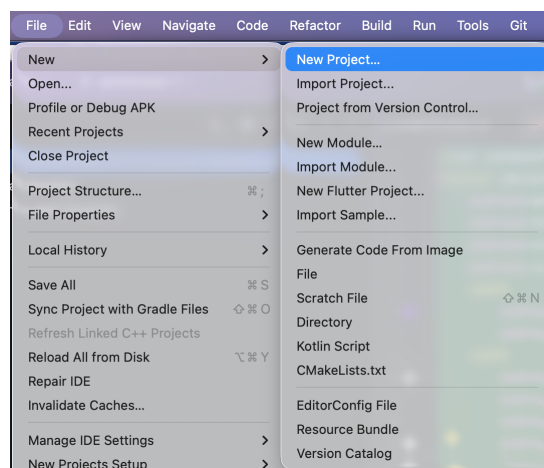
Contoh Implementasi SDK :

1. **Aplikasi Peta** : menggunakan API Google Maps dari SDK.
2. **Aplikasi Kamera** : memanggil Camera API dari SDK.
3. **Aplikasi Lokasi** : memanfaatkan LocationManager dari SDK.

G. Membuat Project Baru pada Android Studio

Pada bagian ini, kita akan memulai untuk membuat proyek pertama mobile Android menggunakan Android studio. Bagi yang menggunakan selain Android Studio, dapat menyesuaikan diri.

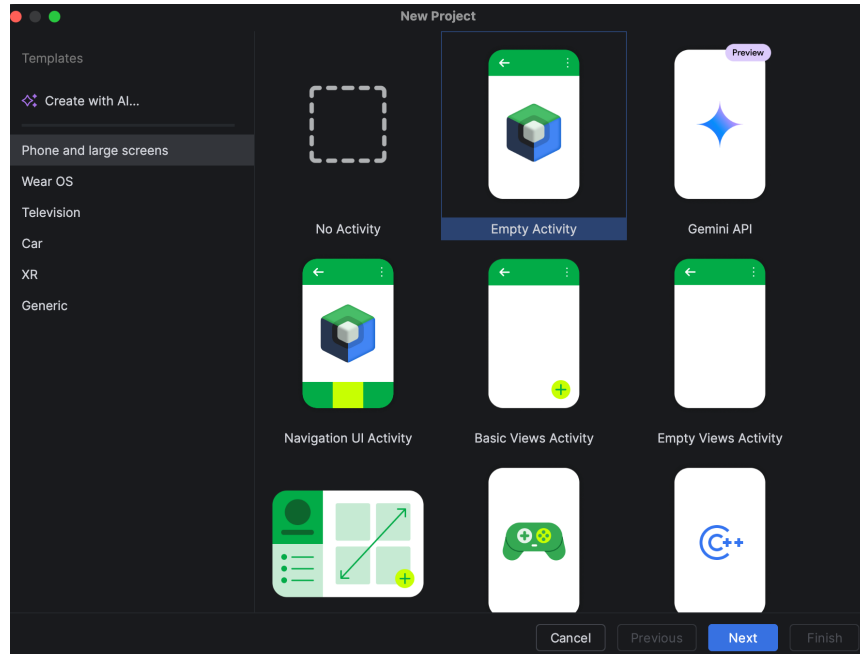
a. Pilih menu File → New Project



Gambar 5. Tampilan Menu Bar Pada Android Studio

a. Pilih “Empty Activity” → Klik tombol “Next”

Empty Activity adalah sebuah *template* (kerangka dasar) bawaan di Android Studio yang digunakan saat membuat proyek baru atau menambahkan layar (halaman) baru ke dalam aplikasi.

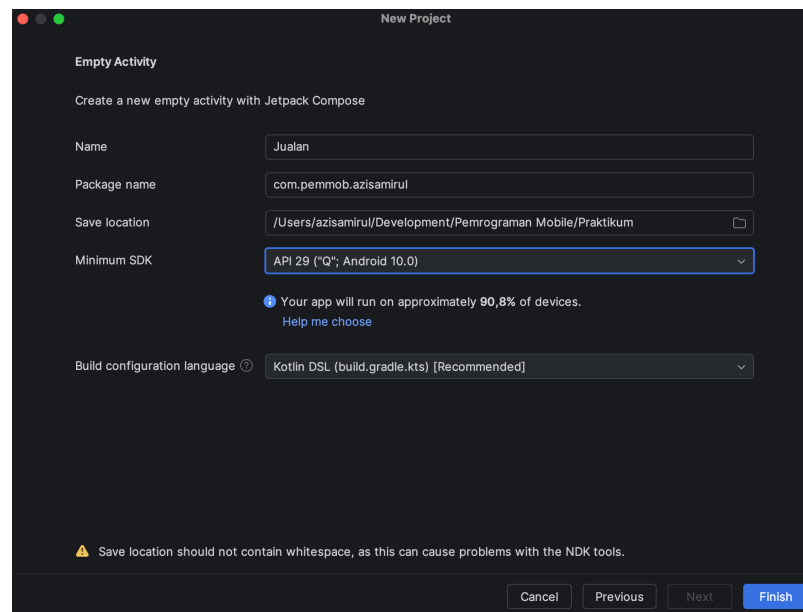


Gambar 6. Tampilan Menu Bar Pada Android Studio

Empty Activity menyediakan struktur kode yang paling minimal dan mendasar. Pengembang akan mendapatkan sebuah layar kosong yang siap diisi, tanpa adanya komponen tambahan bawaan seperti menu navigasi bawah (*bottom navigation*), panel samping (*navigation drawer*), atau tombol melayang (*floating action button*) yang terdapat di *template* lain. Pada konfigurasi di atas, terdapat 2 activity yang mirip yaitu: **Empty Activity** dan **Empty View Activity**. **Empty Activity**: berbasis **Jetpack Compose** (menggunakan Kotlin murni tanpa berkas XML). Sedangkan **Empty Views Activity** adalah nama baru untuk *template* yang implementasi layoutnya masih menggunakan **XML**.

b. Mengisi Kolom nama aplikasi, application id (package name), minimum SDK.

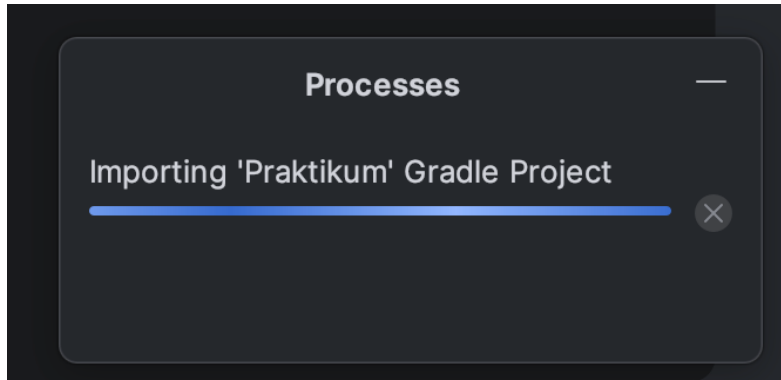
Pada bagian ini, silakan isikan nama aplikasi yang Bernama: “Jualan” dengan package name: **com.pemmob.{{username peserta}}** , contoh: **com.pemmob.anakbaik**.



Gambar 7. Konfigurasi Isian pada Project Aplikasi Android

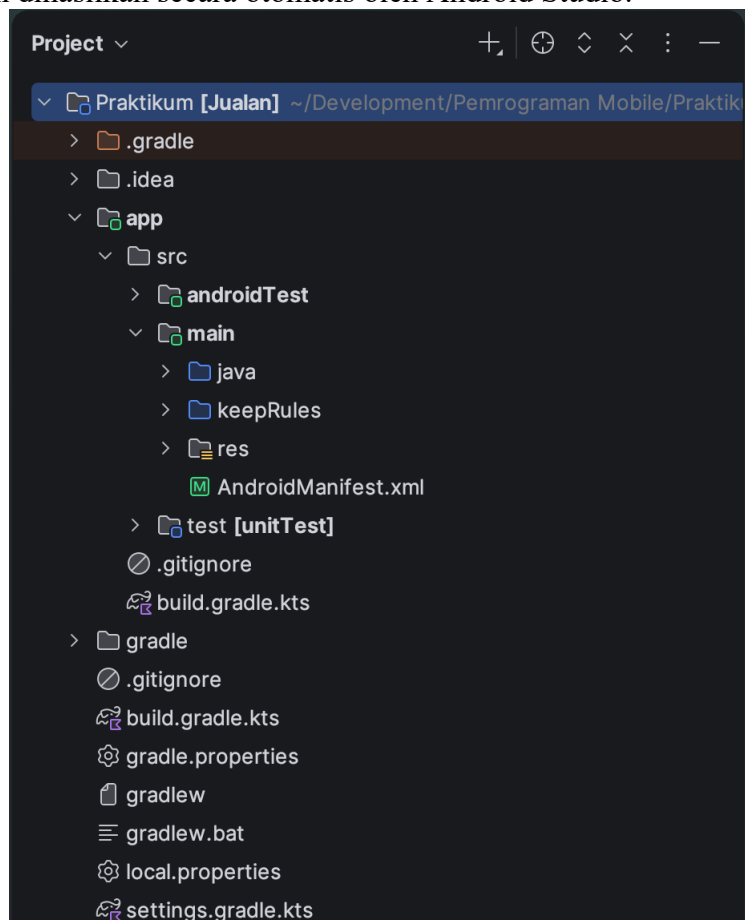
Penjelasan:

1. **Application Name:** Nama aplikasi yang akan ditampilkan kepada pengguna, pada praktikum ini application name adalah: **Jualan**.
 2. **Package Name:** Identitas unik aplikasi Android yang biasanya menggunakan format *reverse domain*, misalnya **com.pemmob.anakbaik**. Package name digunakan untuk membedakan aplikasi kita dengan aplikasi lain, terutama saat dipublikasikan ke Google Play.
 3. **Minimum SDK:** Versi minimum Android yang diperlukan agar aplikasi dapat diinstal dan dijalankan. Misalnya, **API 29 (Android 10)** berarti aplikasi hanya dapat dijalankan pada perangkat dengan Android 10 atau versi yang lebih baru.
- c. Kemudian klik tombol: “Finish”** dan tunggu beberapa saat hingga indikator *loading bar* selesai memproses.



Gambar 8. Proses Inisialisasi

Setelah proses *loading* selesai, pada tool window sebelah kiri terdapat struktur project yang telah dihasilkan secara otomatis oleh Android Studio.



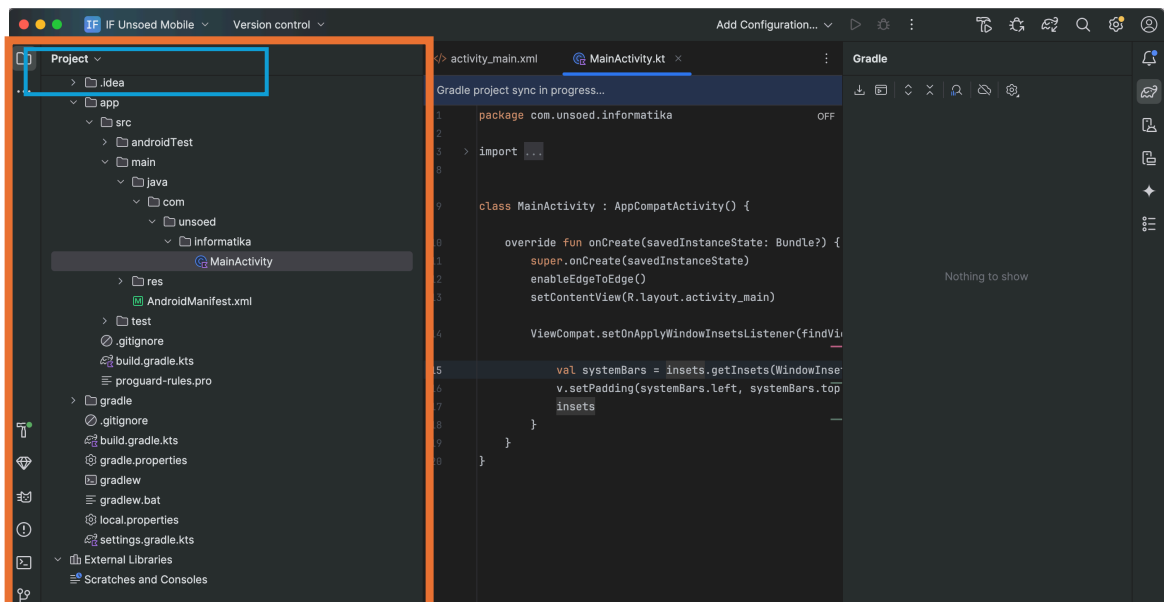
Gambar 9. Struktur File dan Folder pada Project Android

Pada saat implementasi kode program, kita akan menuliskannya pada folder yang terdapat folder: `app/src/main/java` Meskipun bernama "java", folder ini adalah tempat di mana seluruh kode sumber bahasa pemrograman kita disimpan, baik itu ditulis menggunakan Java maupun **Kotlin**.

- **Struktur Paket (*Package*):** Kode diurutkan berdasarkan nama paket (*package name*) yang Anda tentukan saat membuat proyek (contoh: `com.unsoed.anakbaik`).
- **Isi Utama:** Di sinilah kita akan menemukan berkas `MainActivity.kt`, fungsi-fungsi `@Composable` untuk Jetpack Compose, serta kelas lainnya yang dibuat.
- **Folder Pengujian (*Test*):** Terdapat juga sub-folder berakhiran (`androidTest`) dan (`test`) yang digunakan khusus untuk menulis kode pengujian otomatis (*unit testing* dan *instrumentation testing*).

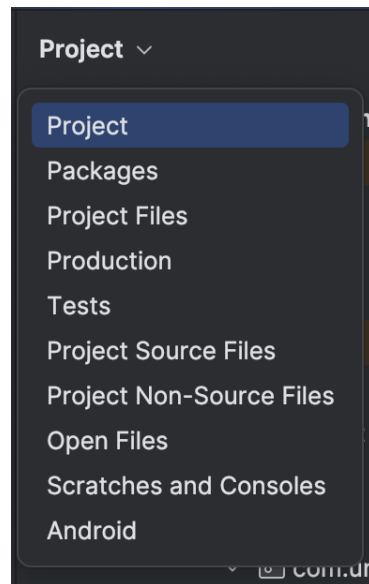
H. Mengenal Struktur Project Android

Pada gambar 10 di bawah ini merupakan tampilan awal ketika project android berhasil dibuat.



Gambar 10. Struktur proyek android

Tampilan struktur proyek pada android dapat diubah sesuai dengan kebutuhan, yaitu dengan cara klik tanda “v” pada bagian atas struktur folder, yang pada gambar 10 ditandai dengan kotak berwarna biru. Pilihan tampilan struktur seperti pada gambar 11 berikut :



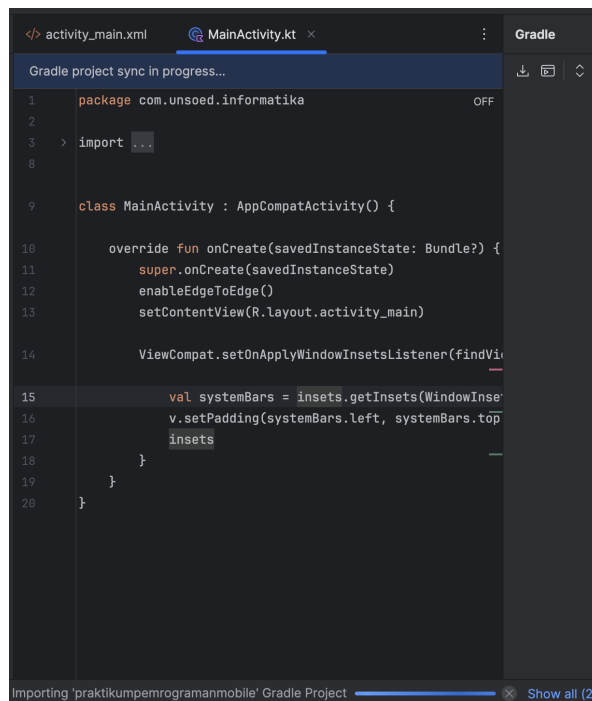
Gambar 11. Pilihan tampilan untuk struktur folder pada proyek android

Berikut adalah penjelasan struktur proyek pada aplikasi android di gambar 10 yang terdapat pada kotak warna oranye :

1. **APP** : Modul utama aplikasi yang berisi seluruh kode, *resource*, dan konfigurasi untuk membangun aplikasi. Pada modul ini terdapat banyak sub-folder antara lain sebagai berikut :
 - `src/` → folder source code maupun resources (font, gambar, dokumen, audio, atau video).
 - `main/` → berisi hal utama aplikasi:
 - `java/` atau `kotlin/` → *source code* yang berkaitan dengan *logic* aplikasi (Activity, Fragment, ViewModel, dsb). Pada saat proyek dibuat dengan pilihan tampilan activity seperti pada gambar 3 di pembahasan sebelumnya, maka secara otomatis Android Studio akan membuat activity yang bernama **MainActivity** dengan format/ekstensi **.kt** karena dibangun dengan Bahasa pemrograman Kotlin.
 - `res/` → resource aplikasi (UI dan aset).
 - `drawable/` → berisi asset berupa gambar/vector.
 - `values/` → berisi nilai/data yang berupa *string*, warna, *style*, dimensi.
 - `mipmap/` → ikon aplikasi.

- `AndroidManifest.xml` → berisi deklarasi komponen aplikasi dan permission/izin akses yang digunakan dalam aplikasi
- `test/` → kode untuk melakukan proses unit test.
- `androidTest/` → kode *instrumented test/automation test* pada android.

Pada saat melakukan inisialisasi proyek android, android studio akan melakukan proses “*Gradle project sync*” yang membutuhkan waktu beberapa detik hingga menit, tergantung dengan kondisi koneksi internet maupun spesifikasi komputer yang digunakan. Seperti yang tertera pada gambar 7 berikut :



Gambar 12. Proses gradle project sync

Gradle merupakan tools untuk melakukan proses *automation build* pada proyek yang menggunakan bahasa pemrograman seperti *java*, *kotlin*, *scala*, atau *groovy*.

Selengkapnya tentang gradle dapat diakses pada tautan berikut: <https://gradle.org/>.

I. Mengganti Icon Aplikasi

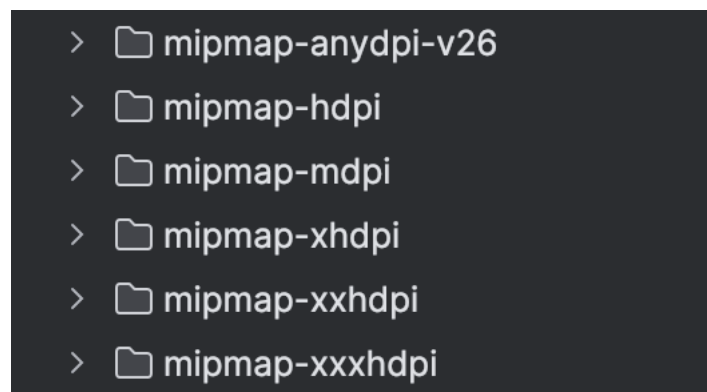
Pada saat aplikasi pertama kali dibuat, maka *icon* aplikasi bawaan adalah logo khas android (robot hijau).



Gambar 13. Icon Default (bawaan) dari Android

Icon aplikasi terletak pada folder res → mipmap. Terdapat beberapa jenis folder mipmap. Dikarenakan ekosistem perangkat Android sangat luas, mulai dari jam tangan pintar berlayar kecil hingga tablet dan televisi beresolusi sangat tinggi. Oleh karena itu, Android menggunakan sistem *Density-Independent Pixel* (dp) dan mengelompokkan folder mipmap berdasarkan kepadatan piksel (*dpi/dots per inch*) layar.

Android telah memberikan panduan mengenai penerapan ikon pada aplikasi yang akan dipublikasikan. Panduan tersebut dapat diakses pada tautan berikut: <https://developer.android.com/distribute/google-play/resources/icon-design-specifications>. Icon aplikasi disimpan pada sub-folder res → mipmap.



Gambar 14. Struktur Folder untuk Icon Aplikasi

Berikut adalah tabel perbandingan jenis folder mipmap dari anydpi hingga xxxhdpi beserta standar ukuran piksel untuk ikon aplikasi:

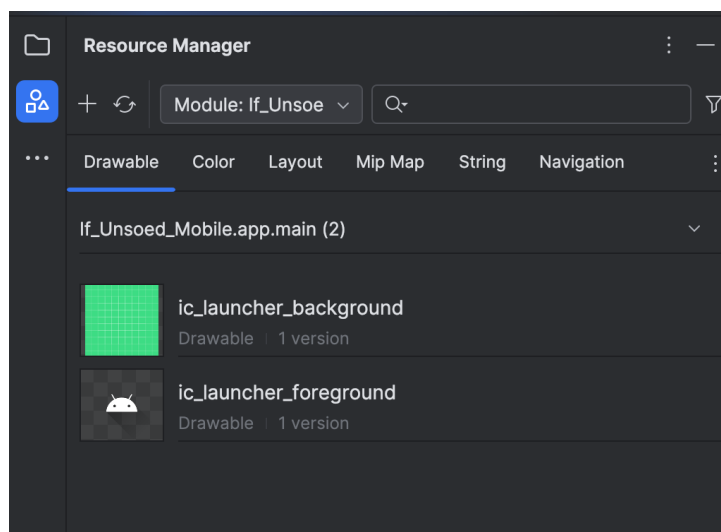
Jenis Folder	Skala Pengali	Kepadatan Layar (DPI)	Ukuran Ikon Standar	Penjelasan
mipmap-anydpi-v26	Fleksibel	Bebas	Tidak terpakai piksel	Folder khusus untuk Adaptive Icons (diperkenalkan sejak Android 8.0/API 26). Menggunakan format vektor (XML). Ikon dapat beradaptasi secara otomatis menjadi bentuk lingkaran, kotak, atau <i>squircle</i> tergantung pengaturan perangkat pengguna.
mipmap-mdpi	1.0x (Dasar)	~160 dpi (Medium)	48 x 48 px	Resolusi dasar/patokan awal. Perangkat lama atau perangkat dengan spesifikasi layar rendah.
mipmap-hdpi	1.5x	~240 dpi (High)	72 x 72 px	Kepadatan tinggi. Standar untuk <i>smartphone</i> kelas bawah hingga menengah generasi lama.
mipmap-xhdpi	2.0x	~320 dpi (Extra High)	96 x 96 px	Kepadatan sangat tinggi. Banyak digunakan pada <i>smartphone</i> kelas menengah standar.
mipmap-xxhdpi	3.0x	~480 dpi (Extra Extra High)	144 x 144 px	Kepadatan ekstra tinggi. Standar umum untuk mayoritas <i>smartphone</i> modern masa kini (layar Full HD).
mipmap-xxxhdpi	4.0x	~640 dpi (Extra Extra Extra High)	192 x 192 px	Kepadatan maksimum saat ini. Digunakan untuk <i>smartphone</i> flagship (layar 2K/4K) dan tablet berkualitas sangat tinggi.

Namun, kita tidak akan mengganti/mengisi icon di folder satu per-satu. Android Studio telah menyediakan fitur untuk mengganti icon aplikasi secara instan. Silakan ikuti langkah – langkah berikut:

a. Menyiapkan asset gambar

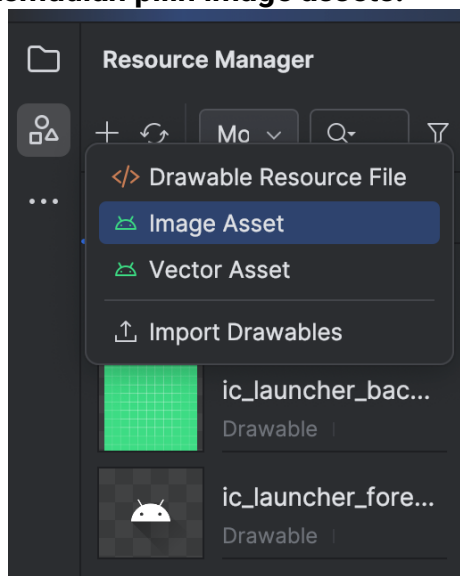
- 1) Silakan unduh gambar “icon_app_jualan.png” pada tautan berikut :
https://drive.google.com/file/d/1-ltVwyEcCh5_pJCNLgxqIT7qfZn_dH-l/view?usp=sharing

b. Pada *tool window* yang terletak di sebelah kiri, silakan pilih menu “Resources Manager”.



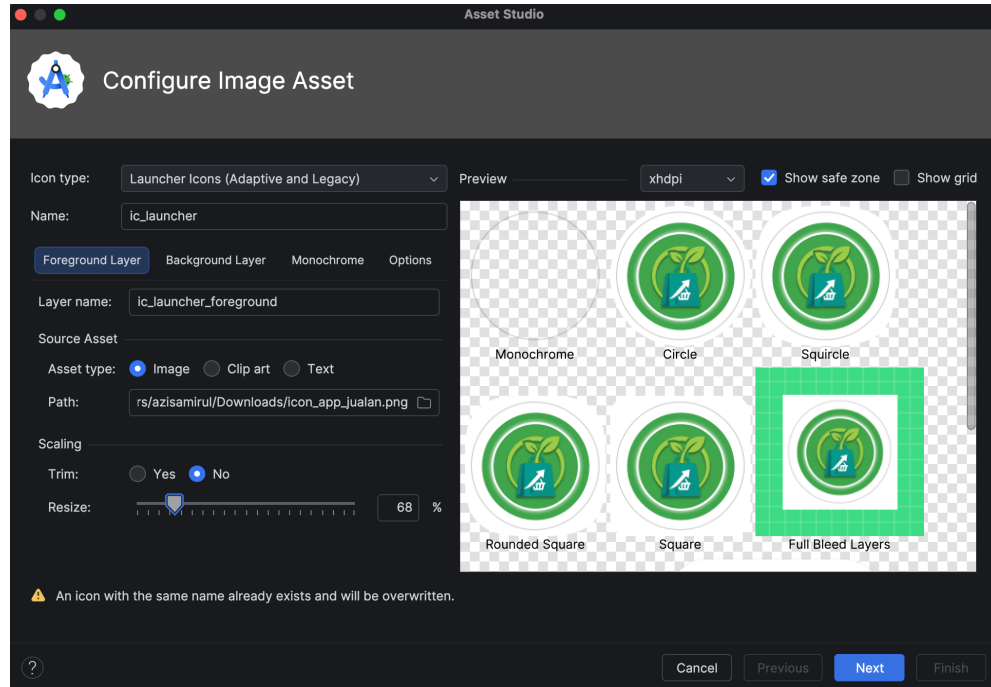
Gambar 15 Tampilan Resources Manager

c. Klik tanda “+” kemudian pilih image assets.



Gambar 16 Menu pada Resources Manager

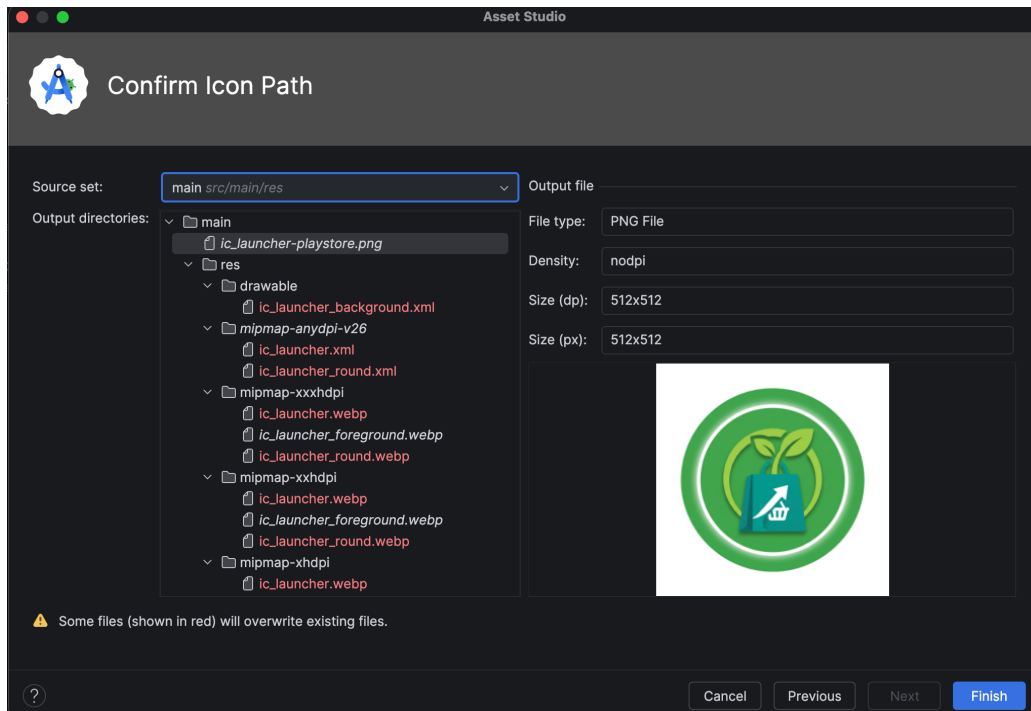
- d. Setelah menu “Image Asset” dipilih, maka akan tampil window “**Configure Image Asset**” seperti pada gambar 13 di halaman selanjutnya. Kemudian silakan klik ikon folder pada kolom “Path” untuk memilih gambar yang telah diunduh pada tahap 1.



- e. Jika gambar telah berhasil dipilih, silakan atur besaran gambar melalui kolom **resize** yang terletak di bagian bawah. Hasil dari *resize* akan muncul di tampilan “Preview” di sebelah kanan yang menunjukkan hasil dari pengaturan besaran, pada tampilan di bagian “*preview*” terdapat berbagai bentuk icon yang ada di perangkat mobile. Klik next jika sudah selesai untuk mengatur besaran icon.

- f. Tampilan berikutnya adalah window/jendela tentang konfirmasi lokasi icon (*icon path*) yang diterapkan yaitu di sub-folder **res → drawable → mipmap**.

Pada bagian *output directories* terlihat bahwa masing – masing folder telah diisi dengan icon yang sebelumnya kita atur di proses nomor 5. Pada bagian ini, silakan klik “Finish”.



Gambar 17. Hasil Penggantian Icon Aplikasi

J. Membuat Function

Pada bagian ini, kita akan memulai untuk membuat function sederhana menggunakan Kotlin. Silakan buka **MainActivity.kt**

a. Mendeklarasikan Function LayoutTentangJualan()

Silakan deklarasikan function baru di bawah function Greeting seperti pada gambar berikut.

```
1 Usage
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

1 Usage
@Composable
fun LayoutTentangJualan() {
}
```

Gambar 18. Deklarasi Function

Kode tersebut merupakan kerangka dasar untuk membuat komponen antarmuka (UI) kustom di Jetpack Compose. Anotasi **@Composable** di bagian awal berfungsi memberitahu sistem Android bahwa fungsi tersebut bukanlah fungsi Kotlin biasa, melainkan fungsi khusus yang bertugas menggambar antarmuka ke layar, sehingga Anda diizinkan untuk memanggil komponen Compose lain di dalamnya. Selanjutnya, **fun LayoutTentangJualan()** adalah deklarasi fungsinya, yang menurut konvensi/aturan Compose wajib diawali dengan huruf kapital (PascalCase) karena fungsi ini bertindak layaknya sebuah komponen UI utuh. Tanda kurung kurawal {} berfungsi sebagai "kanvas" tempat pengembang menyusun berbagai elemen visual, seperti teks, gambar, maupun tata letak Column, yang akan membentuk tampilan halaman aplikasi Anda.

b. Menambahkan Kolom

Di dalam function **LayoutTentangJualan()** tambahkan column seperti pada gambar 19 berikut.

```
Column(  
    modifier = Modifier  
        .fillMaxSize()  
        .padding(all = 16.dp),  
    horizontalAlignment = Alignment.CenterHorizontally  
) {  
  
}
```

Gambar 19. Source Code Column

Penjelasan kode:

1. `Column { }`: komponen *layout* dasar di Compose. Tugas utamanya adalah “menumpuk elemen-elemen” di dalamnya (anak/ *children*) secara **vertikal** dari atas ke bawah.
2. `modifier = Modifier`: Di Jetpack Compose, `Modifier` ibarat alat rias dan pengatur tata letak. Pengembang menggunakannya untuk mengubah ukuran, warna latar belakang, jarak, hingga mendeteksi klik atau *scroll*.
3. `.fillMaxSize()`: Fungsi ini memberitahu `Column` untuk mengambil seluruh ruang kosong yang tersedia dari induknya (baik lebar maupun tingginya).
4. `.padding(16.dp)`: Ini memberikan jarak (ruang kosong) sebesar 16 *density-independent pixels* di bagian dalam wadah, tepatnya di sisi atas, bawah, kiri, dan kanan. Ini mencegah elemen di dalamnya menempel langsung ke ujung layar ponsel.
5. `horizontalAlignment = Alignment.CenterHorizontally`: Aturan ini memberitahu `Column` tentang bagaimana cara mengatur elemen-elemen di dalamnya dari sisi kiri-kanan (horizontal). Perintah ini memastikan semua elemen (seperti teks atau tombol yang nanti Anda masukkan) akan otomatis berada di tengah layar.

c. Menambahkan Box:

Di dalam Column yang telah dibuat, tambahkan Box seperti pada

```
Column(  
    modifier = Modifier  
        .fillMaxSize()  
        .padding(all = 16.dp),  
    horizontalAlignment = Alignment.CenterHorizontally  
) {  
    Box(  
        modifier = Modifier  
            .size(100.dp)  
            .clip(CircleShape)  
            .background(Color.Gray),  
        contentAlignment = Alignment.Center  
    ) {  
  
    }  
  
}
```

Gambar 20. Source Code Implementasi Box

Box adalah salah satu tata letak (*layout*) dasar di Jetpack Compose. Jika Column menyusun elemen ke bawah, Box menyusun elemen **bertumpuk dari belakang ke depan** (seperti lapisan *layer*).

Penjelasan kode:

1. `modifier = Modifier`

Sama seperti sebelumnya, fungsi `modifier` adalah untuk memberikan *gaya/style*, ukuran, dan dekorasi pada `Box`.

2. `.size(100.dp)`

Perintah ini mengatur lebar (*width*) sekaligus tinggi (*height*) dari `Box` menjadi 100dp. Bentuk dasarnya saat ini adalah sebuah persegi (kotak) sempurna.

3. `.clip(CircleShape)`

Bagian ini berfungsi untuk **memotong** (*clipping*) bentuk kotak tadi menjadi sebuah lingkaran sempurna (`CircleShape`). Apa pun yang berada di luar batas lingkaran ini tidak akan ditampilkan.

4. `.background(Color.Gray),`

memberikan warna latar belakang abu-abu pada `Box`.

Catatan untuk urutan Modifier: Karena `.background` dipanggil **setelah** `.clip(CircleShape)`, maka warna abu-abunya hanya akan mengisi area yang sudah dipotong menjadi lingkaran. Jika urutannya dibalik (`background` dulu, baru `clip`), akan terlihat kotak abu-abu utuh (karena warnanya digambar sebelum dipotong).

5. `contentAlignment = Alignment.Center`

Aturan ini mengatur posisi konten (anak-anak) di dalam `Box`. Perintah ini memastikan bahwa apa pun yang Anda masukkan ke dalam `Box` (misalnya ikon atau teks) akan diletakkan **tepat di tengah-tengah** (baik secara vertikal maupun horizontal).

6. { ... }

Merupakan "isi" dari Box. Di dalam kurung kurawal inilah Anda akan meletakkan elemen lain, misalnya sebuah gambar profil atau sebuah huruf inisial nama.

d. Menambahkan Text dalam Box:

Setelah *source code* Box diimplementasikan, selanjutnya kita akan menambahkan Text di dalam Box. Text berfungsi untuk **menampilkan sebuah tulisan ke layar aplikasi**.

```
Box(  
    modifier = Modifier  
        .size(100.dp)  
        .clip(CircleShape)  
        .background(Color.Gray),  
    contentAlignment = Alignment.Center  
) {  
    Text("Jualan", color = Color.White, fontWeight = FontWeight.Bold)  
}
```

Gambar 21. Source Code Implementasi Text

Penjelasan kode:

1. **Text(...)**: Komponen dasar di Compose yang bertugas mencetak teks.
2. **"Jualan"**: Ini adalah isi teks atau kata yang akan dimunculkan ke layar. **Teks wajib diapit oleh tanda kutip ganda ("").**
3. **color = Color.White**: Parameter ini mengubah warna teks menjadi putih.
4. **fontWeight = FontWeight.Bold**: Parameter ini mengatur ketebalan huruf (tipografi). Perintah ini akan membuat tulisan dicetak **tebal (bold)**.

e. Menambahkan **Spacer** di bawah elemen **BOX**

Spacer berfungsi untuk memberikan ruang kosong (jarak) antar elemen di dalam tata letak aplikasi.

```
Box(  
    modifier = Modifier  
        .size(200.dp)  
        .clip(CircleShape)  
        .background(Color.Gray),  
    contentAlignment = Alignment.Center  
) {  
    Column() {  
        Image(  
            painter = painterResource(id = R.drawable.ic_launcher_round),  
            contentDescription = "Jualan".toString(),  
            modifier = Modifier.size(150.dp),  
            contentScale = ContentScale.Crop  
        )  
  
        Text("Jualan", color = Color.White, fontWeight = FontWeight.Bold)  
    }  
}  
  
Spacer(modifier = Modifier.height(24.dp))
```

Gambar 22. Source Code Implementasi Text

Penjelasan kode:

1. **Spacer(...)**: komponen bawaan Jetpack Compose yang bentuknya tidak terlihat (transparan). Tugas utamanya murni hanya untuk mengambil ruang/tempat di layar.
2. **modifier = Modifier.height(24.dp)**: Parameter ini mengatur seberapa besar ruang kosong yang ingin Anda buat. Di sini, Anda menginstruksikan Spacer untuk memiliki tinggi (*height*) sebesar 24dp secara vertikal.

- f. Menambahkan Text “Tentang Jualan” di bawah Spacer dan menambahkan kembali Spacer di bawahnya.

Pada bagian ini kita akan menambahkan teks untuk menampilkan tulisan “Tentang Jualan” serta memberikan jarak di bawah elemen teks tersebut.

```
Text(  
    text = "Tentang Jualan",  
    fontSize = 24.sp,  
    fontWeight = FontWeight.Bold  
)  
  
Spacer(modifier = Modifier.height(16.dp))
```

Gambar 23. Source Code Implementasi Text dan Spacer

- g. Menambahkan konten Text tentang aplikasi serta Spacer

```
Text(  
    text = "Tentang Jualan",  
    fontSize = 24.sp,  
    fontWeight = FontWeight.Bold  
)  
  
Spacer(modifier = Modifier.height(16.dp))  
  
Text(  
    text = "Aplikasi Jualan adalah platform yang mewadahi produk lokal UMKM di wilayah Kabupaten Purbalingga, Jawa Tengah.",  
    fontSize = 16.sp,  
    modifier = Modifier.padding(horizontal = 16.dp)  
)  
  
Spacer(modifier = Modifier.height(32.dp))
```

Gambar 24. Source Code Implementasi Text tentang aplikasi dan Spacer

h. Menambahkan Row di bawah Spacer

Row adalah komponen tata letak (*layout*) dasar yang berfungsi untuk menyusun elemen-elemen di dalamnya secara **horizontal atau sejajar menyamping dari kiri ke kanan**.

A screenshot of a code editor showing the implementation of a Row widget in Kotlin. The code is as follows:

```
Row(  
    modifier = Modifier  
        .fillMaxWidth()  
        .background(Color(0xFFE0E0E0))  
        .padding(all = 16.dp)  
) {  
    // Child elements would go here  
}
```

Gambar 25. Source Code Implementasi Row

Penjelasan kode:

1. **Row { }**: Ini adalah komponen yang berfungsi untuk menata elemen-elemen di dalamnya (anak/ children) secara horizontal (dari kiri ke kanan).
2. **modifier = Modifier**: Sama seperti sebelumnya, modifier pembuka untuk memberikan gaya dan aturan tata letak pada wadah Row tersebut.
3. **.fillMaxWidth()**: Perintah ini akan membuat Row mengambil **seluruh sisa ruang secara mendatar (lebar/ width)** yang tersedia di layar, mengisi penuh dari ujung kiri hingga ujung kanan layar.
4. **.background(Color(0xFFE0E0E0))**: Berfungsi untuk memberikan warna latar belakang pada Row. Pengembang menggunakan kode warna heksadesimal (*Hex Code*). Nilai 0xFFE0E0E0 akan menghasilkan warna **abu-abu terang**. Awalan 0xFF menandakan bahwa warna tersebut 100% solid (tidak transparan).
5. **.padding(16.dp)**: memberikan jarak (ruang kosong) di bagian dalam sebesar 16dp pada semua sisi (atas, bawah, kiri, kanan).

Catatan: .padding diletakkan **setelah** .background. Sehingga, warna latar belakang abu-abu terang tadi akan ikut menutupi area *padding* tersebut. Jika

susunan kode `.padding` sebelum `.background`, maka area *padding* (pinggirannya) tidak akan ikut terwarnai.

i. Menambahkan Text di dalam Row

```
Row(  
    modifier = Modifier  
        .fillMaxWidth()  
        .background(Color(0xFFE0E0E0))  
        .padding(all = 16.dp)  
    ) {  
        Text(  
            text = "Misi Kami:",  
            fontWeight = FontWeight.Bold,  
            modifier = Modifier.weight(1f)  
        )  
        Text(  
            text = "Memajukan UMKM Lokal",  
            modifier = Modifier.weight(2f)  
        )  
    }
```

Gambar 26. Implementasi Text dalam Row

Penjelasan kode:

1. **Modifier.weight()** adalah alat ukur yang digunakan untuk membagi sisa ruang kosong di layar secara proporsional (berdasarkan rasio/perbandingan). **Modifier.weight()** HANYA BISA digunakan di dalam tata letak Row atau Column.
2. **Modifier.weight(1f)**: Pada teks "Misi Kami:", pengembang memberikan `weight(1f)`. Huruf `f` berarti *Float* (tipe data angka desimal di Kotlin). Fungsinya adalah untuk memberitahu *Compose*, "berikan jatah ruang sebesar **1 bagian** untuk teks ini."
3. **Modifier.weight(2f)**: Pada teks "Memajukan UMKM Lokal", pengembang memberikan `weight(2f)`. Fungsinya adalah memberitahu *Compose*, "berikan jatah ruang sebesar **2 bagian** untuk teks ini."

Sistem Jetpack Compose akan menjumlahkan total bobot/*weight* di dalam Row tersebut. Total *weight* = $1f + 2f = 3f$ (**3 bagian total**). Kemudian, layar akan dibagi secara proporsional:

- Teks pertama mendapat **1/3 (sekitar 33%)** dari lebar Row.
- Teks kedua mendapat **2/3 (sekitar 66%)** dari lebar Row.

j. Modifikasi function onCreate()

Pada bagian ini, kita akan memanggil fungsi `LayoutTentangJualan()` yang telah diimplementasi kodennya. Silakan ubah pada bagian dalam `Scaffold` menjadi seperti pada gambar berikut:

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        JuaLanTheme {
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->|
                LayoutTentangJualan()
            }
        }
    }
}

```

Gambar 27. Modifikasi kode pada function onCreate()

Penjelasan kode:

1. Override fun onCreate(...) & super.onCreate(...):

override fun onCreate(...) adalah fungsi utama yang otomatis dijalankan sistem Android saat sebuah halaman aplikasi pertama kali diciptakan atau dibuka, sedangkan **super.onCreate(...)** di dalam fungsi tersebut bertugas menjalankan instruksi bawaan sistem Android yang asli untuk memastikan siklus hidup dasar halaman berjalan dengan benar dan stabil sebelum menambahkan kode /pemanggilan *function* lain. **onCreate(...)** merupakan bagian dari *liveCycle*/siklus hidup pada Activity android. **Lifecycle** (Siklus Hidup) dalam *Android* adalah rangkaian tahapan atau fase yang dialami oleh sebuah komponen aplikasi (seperti halaman layar atau Activity) sejak pertama kali diciptakan hingga akhirnya “dihancurkan” dan “dihapus” dari memori/RAM. Jika diibaratkan, *Lifecycle* adalah "perjalanan hidup" sebuah aplikasi: mulai dari lahir, bangun, beraktivitas, istirahat, hingga

akhirnya mati. Memahami *Lifecycle* (Siklus Hidup) sangat penting karena *smartphone* memiliki keterbatasan yaitu perlunya optimalisasi penggunaan **baterai** maupun **memori (RAM)**. Silakan pelajari secara mendalam tentang *lifecycle* pada tautan berikut:
<https://developer.android.com/guide/components/activities/activity-lifecycle?hl=id>

2. **enableEdgeToEdge()**: Perintah ini membuat aplikasi dapat tampil hingga menembus ke area *status bar* (tempat jam dan baterai di atas) dan *navigation bar* (tombol navigasi di bawah).
3. **setContent { ... }**: `setContent` adalah blok untuk mengimplementasikan UI menggunakan fungsi-fungsi `@Composable`.
4. **JualanTheme { ... }**: Tema khusus aplikasi yang otomatis terbuat saat membuat *project* aplikasi, dengan penamaan namanya menyesuaikan nama *project*. Komponen ini mengatur palet warna (mode gelap/terang), jenis *font* (tipografi), dan bentuk komponen (lengkungan tombol, dll) agar seragam di seluruh aplikasi.
5. **Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding**
-> `Scaffold` adalah kerangka tata letak standar dari *Material Design* untuk mengimplementasikan kode – kode layout.
6. **LayoutTentangJualan()**: fungsi `@Composable` yang telah dibuat untuk menampilkan elemen aplikasi.

Setelah kode selesai diimplementasikan, maka hasil akhir dari aplikasi sederhana yang dibuat adalah sebagai berikut:



Gambar 28. Hasil Implementasi Praktikum Pertemuan ke-1

K. Menjalankan aplikasi

Proses untuk menjalankan aplikasi yang telah dirancang dapat melalui 2 cara, yaitu menggunakan *emulator* atau melalui *USB Debugging* pada perangkat android yang kita miliki.

1. Pilihan 1: Menjalankan aplikasi melalui emulator

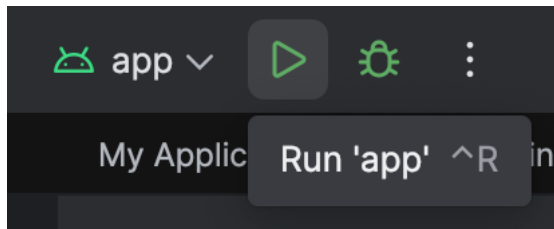
Emulator android merupakan perangkat virtual yang disediakan Android Studio. Developer dapat melakukan simulasi ke perangkat virtual pada aplikasi Android Studio. Namun hal yang perlu diperhatikan sebelum menjalankan *emulator* adalah spesifikasi minimum RAM yang diperlukan yaitu direkomendasikan sebesar 16GB. Jika spesifikasi perangkat komputer Anda memungkinkan, silakan menuju ke tautan berikut untuk mempersiapkannya: <https://developer.android.com/studio/run/managing-avds> .

2. Menjalankan aplikasi melalui perangkat

Jiika spesifikasi komputer Anda tidak memungkinkan, maka alternatif kedua yang dapat dilakukan adalah menjalankan aplikasi pada perangkat *smartphone* Anda, jika Anda pertama kali dalam melakukan ini, silakan mengikuti tahapan berikut dan klik tautan sesuai tahapan:

- Mengaktifkan *developer mode*:
<https://developer.android.com/studio/debug/dev-options#enable>
- **Alternatif 1** : Menggunakan kabel data dengan mode *usb debugging*:
<https://developer.android.com/studio/debug/dev-options#debugging>
- **Alternatif 2**: Menggunakan wifi Debugging:
<https://developer.android.com/studio/run/device#wireless>

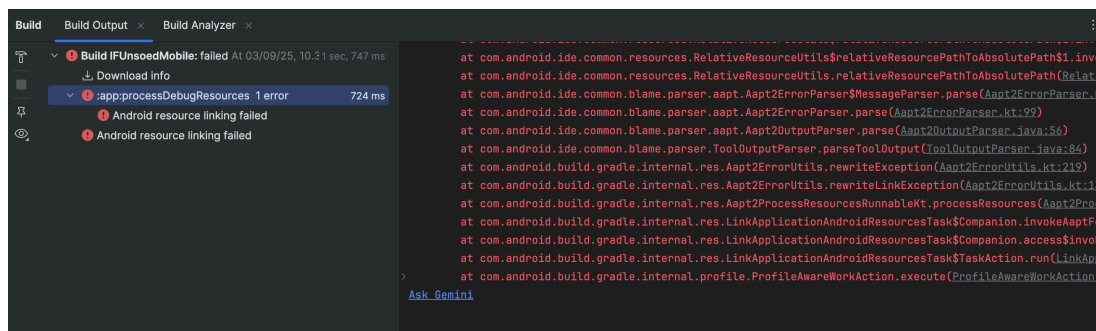
Jika emulator atau perangkat telah siap untuk menjalankan aplikasi, langkah terakhir adalah proses “run” dengan klik tombol seperti pada gambar 33 berikut ini :



Gambar 29. Tombol “Run ‘app’”

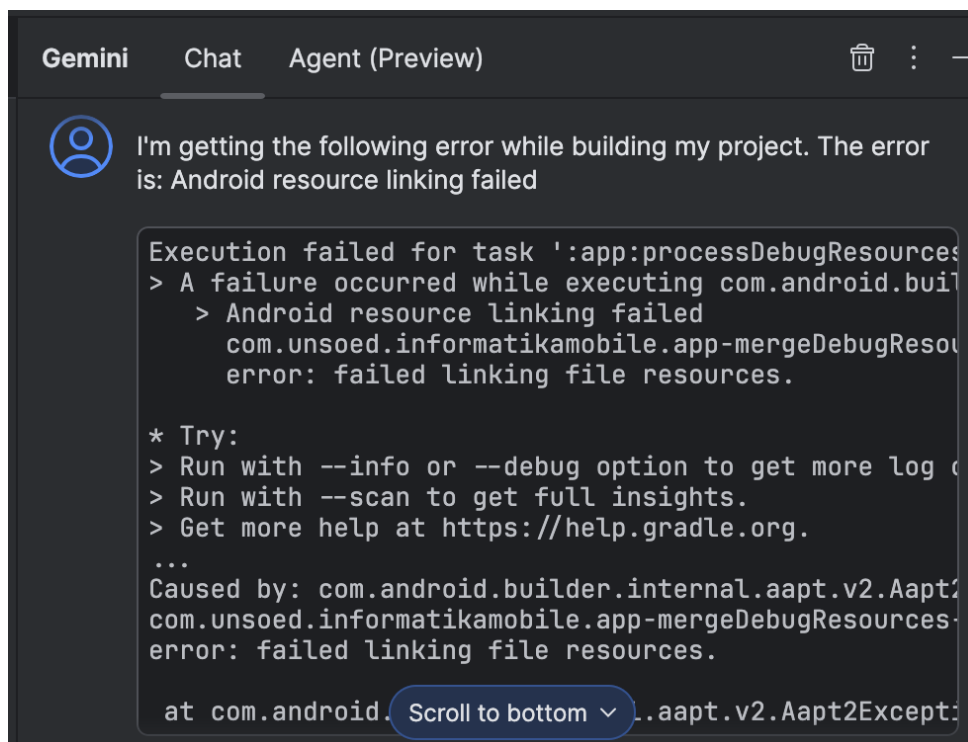
L. Bantuan Artificial Intelligence (AI) Gemini untuk Membantu Memecahkan Masalah Penulisan Kode

Pada saat proses build, terkadang terjadi error yang disebabkan oleh penulisan kode yang tidak sesuai dengan aturan penulisan. Pemberitahuan error tersebut muncul pada status bar yang terletak di posisi bawah.



Gambar 30. Error output pada status bar Android Studio

Android Studio memberikan fasilitas bantuan AI Gemini untuk membantu *developer* dalam menangani error penulisan kode. Hal ini akan membantu *developer* untuk meningkatkan produktivitasnya karena akan menghemat waktu dalam penyelesaian masalah. Prosedur aktivasi Gemini pada Android Studio dapat dilakukan dengan mengikuti petunjuk pada tautan berikut: <https://developer.android.com/studio/gemini/get-started?hl=id>. Fitur ini akan memudahkan *developer* dalam menyelesaikan permasalahan error yang terjadi dan tentu akan menghemat waktu dibandingkan dengan mencari jawaban di *search engine/browsing*.



Gambar 31. Jawaban dari Gemini AI